Hackathon Thessaloniki 2026

Technical Presentation

6 March 2026

Agenda

- General Notes for All Tracks
- Stream 1 Details

  Designing Systems with GenAI: From Requirements to Architecture
- Stream 2 Details

  AI-Powered Knowledge Base System
- Stream 3 Details

  AI-Powered Cybersecurity Control Centre

General Notes for All Tracks

Environment details

The final product to be running inside a Docker container in a Linux environment

If you need any database use only official images from dockerhub

If external data are needed:

Use Officially provided public APIs (e.g., Wikipedia, OpenAI, HuggingFace, etc.), or mock data that you've generated or curated.

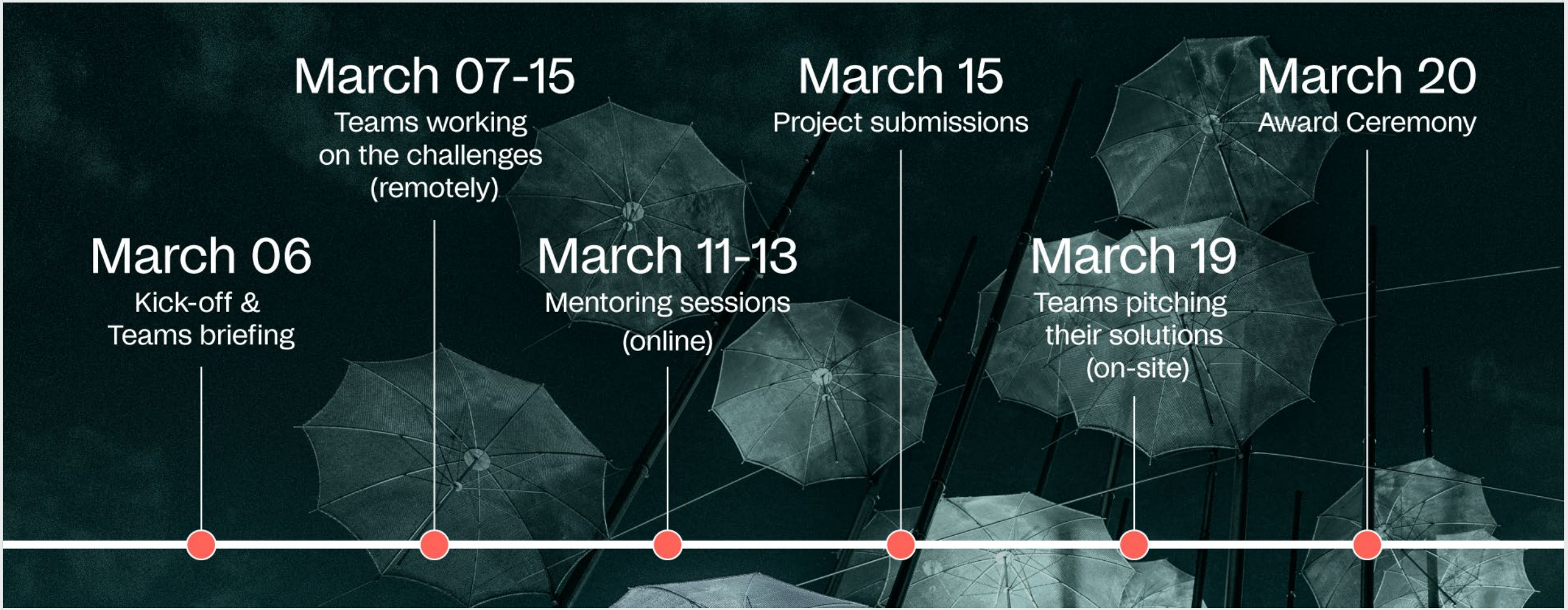
Include a README file containing:

Brief concept description of the system

Setup / Run Instructions

Summary of main features and architecture

Hackathon Timeline



Expected Submissions & Deliverables

Submission details

- Resources
  - After kickoff, check your emails for further information regarding your personalized
    OpenAI API keys, endpoint details and additional resource links
- Submission Instructions
  - Upload your complete and working code to our GitHub repo, following the provided repository guidelines.
  - Don't forget to include a README file that explains the concept, features, and any extra installation instructions needed.
- Best Practices & Restrictions
  - Ensure secure management of API keys and proper Docker containerization.
  - Code is expected to compile and run cross-platform. If docker cannot compile and start successfully,
    or is not easily fixed, it will be considered void
- A short presentation of the solution would be welcome, although not mandatory (video, power point or both). It will not be counted against score
  - Optionally include a short presentation (private Youtube video link, PowerPoint, or both) explaining your solution.
- Pitching

Scoring Criteria

Scoring Criteria – to be updated

Area	Criteria	Description	Area Weight
Innovation	• Fit Solution • Novelty • Creativity • Out-of-the-box thinking	Evaluate the originality, creativity, and problem-solving effectiveness of the solution. Did the group come up with a fresh idea and creative solution that effectively addresses the problem and fits the context?	25%
Potential Impact / Business Value	• Market need • Applicability • Feasibility • Future Usage	Evaluate whether the project demonstrates clear value, scalability, and potential real-world applicability of the project or market adoption. Does the solution have a meaningful and measurable impact on the target audience or industry?	15%
Technical execution	• Working State • Code Quality • Optimization & Efficiency • Modern Tools & Techniques	Evaluate the use of technology, the complexity of the solution and the functionality of the prototype. Are the delivered artifacts functional and reflective of the project's objectives?	25%
UX / Ease of use	• Cross-Platform Functionality • Cross-Device Functionality • Accessibility • Efficiency & Intuitiveness • Engagement & Stickiness	Consider the design, accessibility, and overall user experience of the solution. Is it functional and user-friendly, without requiring additional training for the end user?	15%
Pitching	• Clarity of Presentation • Storytelling & Narrative • Quality • Q&A Handling • Time Management	Evaluate the team's ability to clearly communicate their vision, problem-solving approach, and technical decisions. Was the demo compelling and well-structured? Did the team handle questions confidently and demonstrate deep understanding of their solution?	20%

Challenge 1

Designing Systems with GenAI: From Requirements to Architecture

Ideation

What you'll build & what we expect

GENAI AS A SYSTEM DESIGNER

Design and implement a GenAI-powered system that acts as a System Designer.

The goal is not to build a simple chatbot, but a solution where GenAI:

- Analyzes business needs & requirements
- Understands constraints & trade-offs
- Produces structured technical design artifacts

GenAI should be used as a thinking and design engine, not merely as a conversational interface.

EXPECTED OUTCOMES

Working Prototype

A functional system that takes business requirements as input and generates design artifacts

Design Artifacts

The system should produce structured outputs: architecture diagrams, component specs, data models, or similar

Live Demo

Present your solution in action — show the end-to-end flow from input to generated design

Approach & Rationale

Explain your design choices, GenAI strategy, and what makes your approach unique

Things to Consider

Guiding questions to help shape your approach - not requirements!

1. What types of systems will your designer handle?

   Web apps? Microservices? Data pipelines? APIs? Or any system?

2. How does the user describe their needs?

   Free-text? Structured forms? Conversational? Upload docs?

3. What design artifacts will be generated?

   Architecture diagrams, ERDs, API specs, deployment plans, tech stack recommendations?

4. How does GenAI reason about trade-offs?

   Scalability vs cost? Security vs speed? How are constraints factored in?

5. Can the user iterate on the design?

   Refine requirements, challenge decisions, explore alternatives?

6. How do you validate the generated designs?

   Consistency or constraint checks? Best practices? Are there feedback / self -critique loops?

This is an open-ended challenge - take it in the direction that excites you!

Tech Toolbox

Ideas & directions for your technology stack - mix and match freely

LLM Providers

OpenAI (GPT), Anthropic (Claude), Google (Gemini), Open-source (Llama, Mistral)

Orchestration & Agents

LangChain, LangGraph, CrewAI, AutoGen, custom agent pipelines, multi-step reasoning

Output & Diagrams

Mermaid.js, PlantUML, Structurizr (C4), D2, GraphViz, OpenAPI/Swagger, JSON/YAML schemas

Knowledge & Context

RAG with vector DBs (Pinecone, Chroma, Weaviate, FAISS, pgvector), design pattern libraries, reference architectures

Frontend & UX

Streamlit, Gradio, React, Next.js - interactive UI beyond a simple chat window

Prompting & Techniques

Chain-of-thought, structured output (JSON / Pydantic), prompt chaining, tool use / function calling, ReAct / Plan-and-Execute patterns

No restrictions on tech stack - use whatever helps you build the best solution

Challenge 2

AI-Powered Knowledge Base System

Ideation

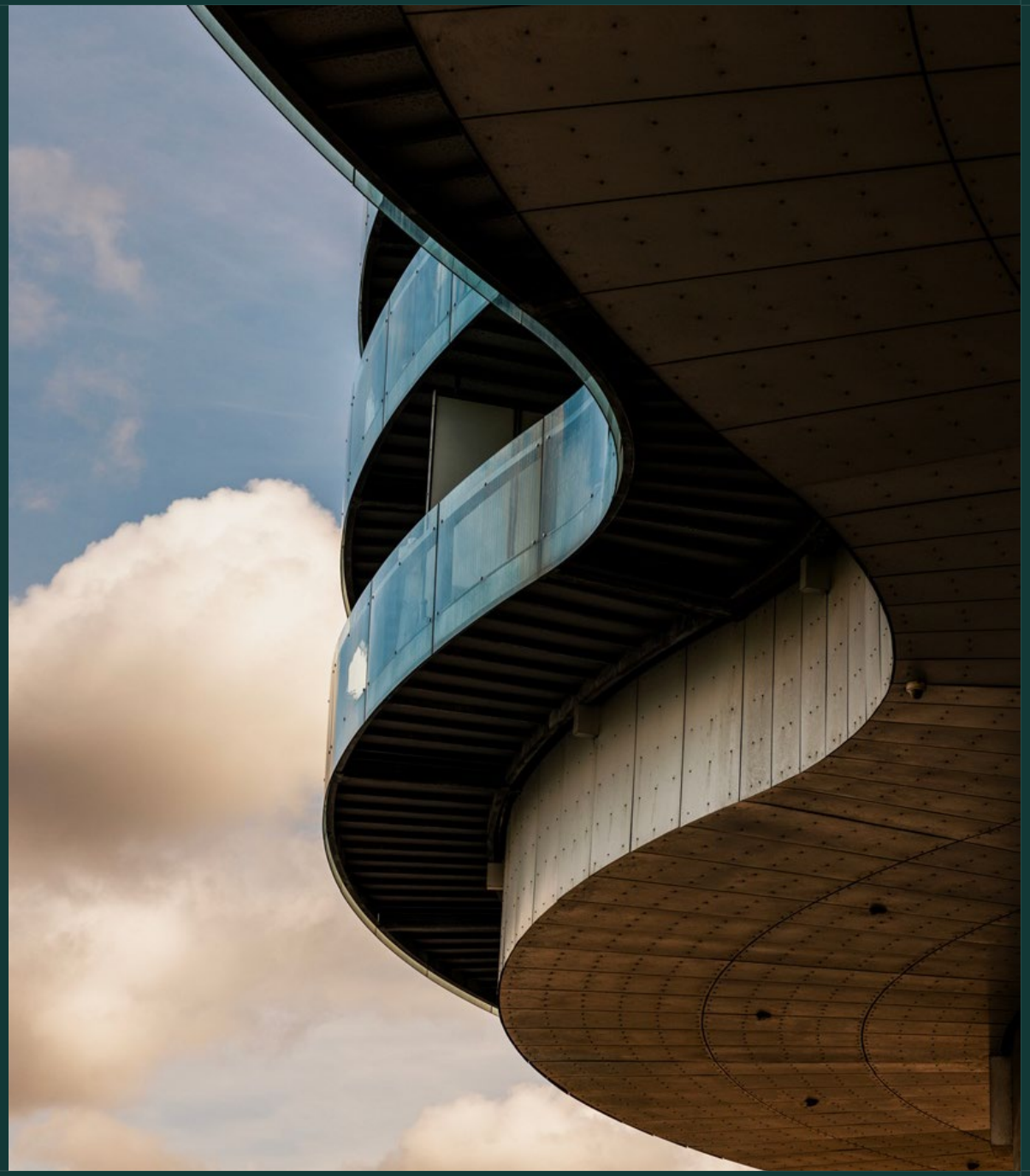
Design and implement an **intelligent Knowledge Base solution** powered by GenAI.

The goal is to go beyond a simple Q&A bot - build a system that understands, organizes, and retrieves knowledge intelligently.

Your solution should leverage AI to:

- Ingest and structure knowledge from documents, APIs, or provided data sources.
- Allow natural language querying with relevant, explainable responses.
- Learn from user interactions, refining relevance or summaries over time.
- Include semantic search or retrieval-augmented generation (RAG) for grounded answers.
- GenAI should act as the reasoning and retrieval engine, not just a text generator - use it to connect, interpret, and synthesize knowledge meaningfully.

To make the problem tangible and show business value, each team should select a specific domain and tailor the knowledge base to that context.



Expected Outcomes

- **Working Prototype:**
  A functioning system that allows users to add, search, and query information using natural language.
- **Knowledge Artifacts:**
  The system should demonstrate structured output - e.g., knowledge graphs, entity relationships, embeddings, or summarized content snippets.
- **Live Demo:**
  Present your solution end-to-end - from ingestion and indexing to natural language answers and reasoning results.
- **Approach & Rationale:**
  Explain your system's architecture, how GenAI is used (indexing, semantic search, summarization, etc.), and **why your design choices matter** (e.g., scalability, modularity, novelty).

Tech Stack

Core Technologies

- Runtime Environment: Dockerized application running on a Linux base image.
- Backend: Indicative choices: Python (FastAPI, Flask) or Node.js (Express, NestJS).
- Frontend (optional): Indicative choices: React, Vue.js, or a simple HTML/JS UI for querying and visualization.
- Database/Search Layer: PostgreSQL, SQLite, or vector databases (e.g., FAISS, Chroma, Weaviate, or Elasticsearch).
- AI / LLM Frameworks: OpenAI API, Hugging Face Transformers, LangChain, or LlamaIndex for retrieval and reasoning.

Supporting Tools

- Containerization: Docker / Docker Compose for multi-service setups.
- Embedding / RAG Pipelines: Use of bm25, TF-IDF, or dense embeddings for semantic retrieval.
- Documentation: Clear README and optional architectural diagram (Mermaid, draw.io, or similar).

Challenge 3

AI-Powered Cybersecurity Control Center

Ideation

Objective

Develop a web app that leverages the OpenAI API to gather intelligence from network defense perimeter devices, such as firewalls or detection and response tools, using RestAPI or any other collection method to provide insights. It needs to ingest the data provided from these resources

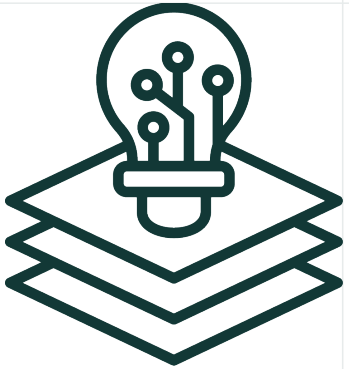
Business Requirements

The solution can be considered as a SoC assistant, it needs to answer questions ie. “show me the latest critical events happening on my infrastructure” or “can you correlate between the events, if there is an attack taking place” or “can you tell me how to block this action”

Web Application

The final product must be a web application running inside a Docker container.

Tech Stack



Proposed/Indicative Technologies

- Programming Language: Python (v 3.8+ recommended)
- Core API: Open AI for integrating AI functionalities (individual keys will be provided after kickoff)
- Frameworks & Libraries: You could use Flask, Django, LangChain or other Python libraries for web development and AI tasks.

Explanation

- Dockerising your solution ensures consistency and portability across all team members, mentors & judges.
- OpenAI API will be central to the autonomous capabilities of your agent.

Notes

- Try to use only Python, Python libraries and the OpenAI API; do not mix in other languages or frameworks

Netcompany

Netcompany.com